Transfer learning

In this notebook we look at how knowledge from a well-explored process (task A) can be transferred to a new process with very few experiments (task B), using the historical PLS model from the previous notebooks.

Notebook setup

Run the next cell first. In Google Colab it clones the course repository, then imports the libraries and the course modules. If you cloned the repository and run the notebook locally (see the README), it only does the imports.

What is a learnable task?

Informally, a task $\mathcal{T}$ is: dataset $\mathcal{D}$, loss function $\mathcal{L} \implies$ model f_θ

with the learning goal $\min_\theta \mathcal{L}(\theta, \mathcal{D})$.

Different tasks can vary based on:

- different objects
- different people
- different objectives
- different conditions
- different ... molecules, processes, products, projects

The bad news: different tasks need to share some structure. If this does not hold, you are better off using single-task learning.

The good news: there are many tasks with shared structure, e.g. the laws of physics that underlie real data, or the behaviour and intentions of people or organisms.

Problem definitions

The multi-task learning problem: learn *a set of tasks* more quickly or more proficiently than learning them independently.

The transfer learning problem: given data on previous task(s), learn *a new task* more quickly and/or more proficiently.

But doesn't multi-task learning reduce to single-task learning?

$$\mathcal{D} = \bigcup_i \mathcal{D}_i \quad \text{and} \quad \mathcal{L} = \sum_i \mathcal{L}_i$$

... Yes, it can! Aggregating the data across tasks and learning a single model is one approach to multi-task learning.

But ...

- What if you want to learn new tasks?
- How do we tell the model which task to do?
- What if aggregating does not work?

How to distinguish between tasks?

Define a **task descriptor** z_i. Instead of $f_\theta(y \mid x)$ we learn $f_\theta(y \mid x, z_i)$.

The task descriptor **can be a one-hot encoding** of the task index, or whatever meta-data you have (personalization using features/attributes, a description, or a formal specification).

The remaining decisions are:

- **the model**: how should the model be conditioned on z_i? Which parameters of the model should be shared?

  Choosing how to condition on z_i is equivalent to choosing how and where to share parameters. E.g. multiplicative gating gives independent training with no shared parameters, concatenation gives all parameters shared except the parameters directly following z_i.

- **the objective**: how should the objective be formed?

  Equally weight all tasks, or weight them differently. Decide manually based on importance or priority, or dynamically adjust the weights during training (various heuristics exist, such as encouraging gradients of similar magnitude in neural networks, or optimizing for the worst-case task loss).

- **the optimization**: how should the objective be optimized?

  Ensure that the tasks are sampled roughly uniformly, regardless of the data quantities. For regression make sure that the *task labels are on the same scale!*

What is transfer learning?

Solve the target task $\mathcal{T}_b$ after solving the source task(s) $\mathcal{T}_a$, by transferring the knowledge learned from $\mathcal{T}_a$.

Transfer learning is a valid solution to multi-task learning, but not vice versa.

Where does transfer learning make sense?

- When $\mathcal{D}_a$ is very large
- When you do not care about solving $\mathcal{T}_a$ and $\mathcal{T}_b$ simultaneously
- Transfer learning via fine-tuning of pretrained models

Import and generate the datasets

In this notebook we use two processes:

- **Task A** is the process explored in the previous notebooks. Its pre-generated dataset of 50 experiments is imported from `datasets/`.
- **Task B** is a *similar but different* process (slightly different kinetic parameters, e.g. a stronger lactate inhibition). Only 10 experiments are available for training and they cover a restricted region of the process conditions. A test set of 100 experiments covering the full design space is generated to evaluate the models.

The target in this notebook is the amount of **aggregates** at the end of the process.

```
""" Task A: previously generated data from which we want to transfer knowledge """
# Define filename to import
FILENAME = "owu.csv"
FILEPATH = "datasets/"
# Import OWU data
A_doe = pd.read_csv(FILEPATH+FILENAME.replace(".csv","_doe.csv"),index_col=None,
usecols =["feed_start","feed_end","Glc_feed_rate","Glc_0","VCD_0"])
A_owu = pd.read_csv(FILEPATH+FILENAME,index_col=None, usecols = ["X:VCD", "X:Glc",
"X:Lac", "X:Titer","W:Feed"])
A_owu.index =
pd.MultiIndex.from_product([list(range(int(len(A_owu)/15))),list(range(15))], names=
["run","time"])
A_bwu = simulator.generate_bwu(A_owu)
A_tar = simulator.generate_y(A_bwu, return_aggr=True)
```

```
""" Task B: new process which we want to model """
# DOE Dataset definition (variable = [lower bound, upper bound])
# Model parameters: Dictate the behaviour of the cell process
MU_G_MAX = 0.05   # 0.05
MU_D_MAX = 0.027  # 0.025
K_G_GLC  = 1      # 1
K_I_LAC  = 20     # 30
K_D_LAC  = 50     # 50
K_GLC    = 0.05   # 0.04
K_LAC    = 0.06   # 0.06
K_PROD   = 1      # 1
""" Train set: restricted region of the process conditions """
# Process parameters: Conditions at which process is run
FEED_START = [1, 4]       # [1, 4]
FEED_END = [8, 12]        # [8, 12]
GLC_FEED_RATE = [10,15]   # [5, 20]
GLC_0 = [30, 60.0]        # [10, 80.0]
VCD_0 = [0.1, 0.5]        # [0.1, 1.0]
# Collect parameters to dictionary
VAR_LIMS = {"mu_g_max":MU_G_MAX, "mu_d_max": MU_D_MAX, "K_g_Glc" : K_G_GLC, "K_I_Lac"
: K_I_LAC, "K_d_Lac" : K_D_LAC, "k_Glc" : K_GLC, "k_Lac" : K_LAC, "k_Prod" : K_PROD,
    "feed_start" : FEED_START,"feed_end" : FEED_END, "Glc_feed_rate" : GLC_FEED_RATE,
"Glc_0" : GLC_0, "VCD_0" : VCD_0}
# Number of experiments to generate
NUM_RUNS = 10
# Filename and filepath for the dataset
FILENAME = "owu_taskB.csv"
FILEPATH = "generated/"
# Generate Dataset
data = simulator.generate_data(VAR_LIMS, NUM_RUNS, FILEPATH + FILENAME)
# Import DOE and OWU
B_doe = pd.read_csv(FILEPATH+FILENAME.replace(".csv","_doe.csv"),index_col=None,
usecols =["feed_start","feed_end","Glc_feed_rate","Glc_0","VCD_0"])
B_owu = pd.read_csv(FILEPATH+FILENAME,index_col=None, usecols = ["X:VCD", "X:Glc",
"X:Lac", "X:Titer","W:Feed"])
B_owu.index = pd.MultiIndex.from_product([list(range(NUM_RUNS)),list(range(15))],
names=["run","time"])
B_bwu = simulator.generate_bwu(B_owu)
B_tar = simulator.generate_y(B_bwu, return_aggr=True)
""" Test set: full design space """
# Process parameters: Conditions at which process is run
FEED_START = [1, 4]       # [1, 4]
FEED_END = [8, 12]        # [8, 12]
GLC_FEED_RATE = [5,20]   # [5, 20]
GLC_0 = [10, 80.0]        # [10, 80.0]
VCD_0 = [0.1, 1.0]        # [0.1, 1.0]
# Collect parameters to dictionary
VAR_LIMS = {"mu_g_max":MU_G_MAX, "mu_d_max": MU_D_MAX, "K_g_Glc" : K_G_GLC, "K_I_Lac"
: K_I_LAC, "K_d_Lac" : K_D_LAC, "k_Glc" : K_GLC, "k_Lac" : K_LAC, "k_Prod" : K_PROD,
    "feed_start" : FEED_START,"feed_end" : FEED_END, "Glc_feed_rate" : GLC_FEED_RATE,
"Glc_0" : GLC_0, "VCD_0" : VCD_0}
# Number of experiments to generate
NUM_RUNS_TEST = 100
# Filename and filepath for the dataset
FILENAME = "owu_taskB_test.csv"
FILEPATH = "generated/"
# Generate Dataset
data_test = simulator.generate_data(VAR_LIMS, NUM_RUNS_TEST, FILEPATH + FILENAME)
# Import DOE and OWU
```

```
B_doe_test = pd.read_csv(FILEPATH+FILENAME.replace(".csv","_doe.csv"),index_col=None,
usecols =["feed_start","feed_end","Glc_feed_rate","Glc_0","VCD_0"])
B_owu_test = pd.read_csv(FILEPATH+FILENAME,index_col=None, usecols = ["X:VCD",
"X:Glc", "X:Lac", "X:Titer","W:Feed"])
B_owu_test.index =
pd.MultiIndex.from_product([list(range(NUM_RUNS_TEST)),list(range(15))], names=
["run","time"])
B_bwu_test = simulator.generate_bwu(B_owu_test)
B_tar_test = simulator.generate_y(B_bwu_test, return_aggr=True)
```

Historical PLS model on the BWU matrix

As in the PLS notebook, the BWU matrix is created and pre-processed:

- Remove the titer, lactate and feed columns
- Remove the days after `USE_DAYS` and the initial day 0
- Eliminate invariant columns
- Add the process parameters (DOE) at the beginning
- Create a PLS model from the resulting matrix to the final aggregates

```
""" Number of days of process history """
USE_DAYS = 10
""" Remove variables """
REMOVE_VARS = ['W:Feed','X:Titer','X:Lac'] # X:Titer, X:Lac, X:Glc, X:VCD. W:Feed
""" Normalize data """
USE_NORM = True
""" Number of latent variables """
N_LV = 6
```

```
def preprocess_bwu(bwu, doe, use_days=USE_DAYS, remove_vars=REMOVE_VARS):
    remove_columns = []
    # Remove variables
    for ivar in ['X:Titer','X:Lac','X:Glc','X:VCD','W:Feed']:
        if ivar in remove_vars:
            remove_columns.extend([c for c in bwu.columns if c.startswith(ivar)])
    # Remove history after use_days and the initial day
    remove_columns.extend([c for c in bwu.columns if c.endswith(':0')])
    for d in range(use_days, 15):
        remove_columns.extend([c for c in bwu.columns if c.endswith(":"+str(d))])
    # Remove invariant columns
    remove_columns.extend(list(bwu.columns[~(bwu != bwu.iloc[0]).any().values]))
    # Add the DOE and remove the columns
    X_preproc = pd.concat([doe, bwu.drop(set(remove_columns), axis=1)], axis=1)
    return X_preproc
```

Single-task model on task B

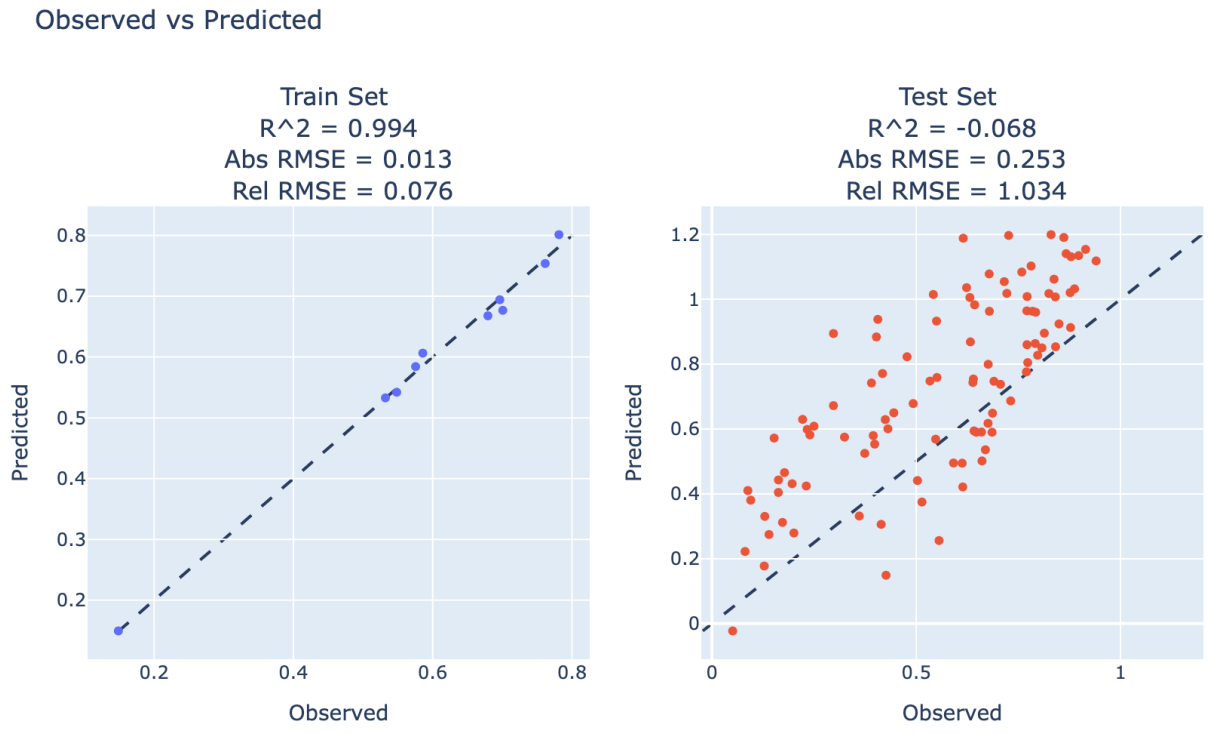
Here we look at the predictive performance of a model trained on the task B data only. It has only 10 experiments available, with the variability of some variables restricted, which makes it hard to generalize to the large test set of 100 experiments.

```
# Define Pipeline
pscaler = StandardScaler(with_mean=USE_NORM,with_std=USE_NORM)
pls_bwu = PLSRegression(n_components=N_LV)
pipe = Pipeline([('scaler', pscaler), ('pls', pls_bwu)])
# Train PLS model
y = B_tar
X = preprocess_bwu(B_bwu, B_doe, USE_DAYS, REMOVE_VARS)
X_columns = X.columns
print(len(X_columns), "input columns:", list(X_columns))
pipe.fit(X,y)

# Make predictions
yhat = pipe.predict(X)
X_test = preprocess_bwu(B_bwu_test, B_doe_test, USE_DAYS, REMOVE_VARS)
y_test = B_tar_test
yhat_test = pipe.predict(X_test)

# Calculate the relative test error for the comparison at the end
scen_1 = round(root_mean_squared_error(y_test, yhat_test) / np.std(np.array(y_test)),
3)
# Plot observed vs predicted with error metrics (R2, absolute and relative RMSE)
plothelpers.observed_vs_predicted(y, yhat, y_test, yhat_test)
```

```
23 input columns: ['feed_start', 'feed_end', 'Glc_feed_rate', 'Glc_0', 'VCD_0',
'X:VCD:1', 'X:VCD:2', 'X:VCD:3', 'X:VCD:4', 'X:VCD:5', 'X:VCD:6', 'X:VCD:7',
'X:VCD:8', 'X:VCD:9', 'X:Glc:1', 'X:Glc:2', 'X:Glc:3', 'X:Glc:4', 'X:Glc:5',
'X:Glc:6', 'X:Glc:7', 'X:Glc:8', 'X:Glc:9']
```



Transfer learning over tasks A and B without one-hot encoding

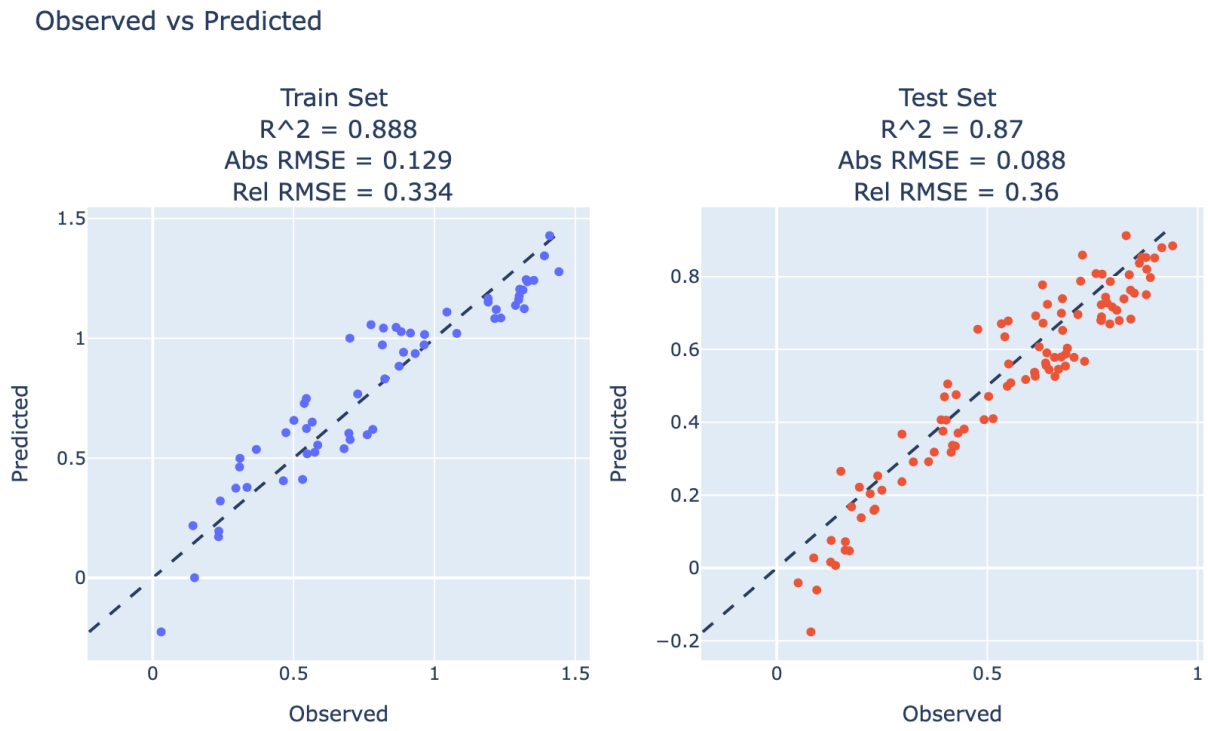
The simplest form of transfer: we pool the data of both tasks and train a single model. The model cannot distinguish the two processes.

```
# Define Pipeline
pscaler = StandardScaler(with_mean=USE_NORM,with_std=USE_NORM)
pls_bwu = PLSRegression(n_components=N_LV)
pipe = Pipeline([('scaler', pscaler), ('pls', pls_bwu)])
# Train PLS model
y = pd.concat([B_tar,A_tar])
X = pd.concat([preprocess_bwu(B_bwu, B_doe, USE_DAYS,
REMOVE_VARS),preprocess_bwu(A_bwu, A_doe, USE_DAYS, REMOVE_VARS)],axis=0)
#X['Task:B'] =
np.concatenate((np.repeat(1,len(B_tar)),np.repeat(0,len(A_tar))),axis=None)
X_columns = X.columns
print(len(X_columns), "input columns:", list(X_columns))
pipe.fit(X,y)

# Make predictions
yhat = pipe.predict(X)
X_test = preprocess_bwu(B_bwu_test, B_doe_test, USE_DAYS, REMOVE_VARS)
#X_test['Task:B'] = np.repeat(1,len(B_tar_test))
y_test = B_tar_test
yhat_test = pipe.predict(X_test)

# Calculate the relative test error for the comparison at the end
scen_2 = round(root_mean_squared_error(y_test, yhat_test) / np.std(np.array(y_test)),
3)
# Plot observed vs predicted with error metrics (R2, absolute and relative RMSE)
plothelpers.observed_vs_predicted(y, yhat, y_test, yhat_test)
```

```
23 input columns: ['feed_start', 'feed_end', 'Glc_feed_rate', 'Glc_0', 'VCD_0',
'X:VCD:1', 'X:VCD:2', 'X:VCD:3', 'X:VCD:4', 'X:VCD:5', 'X:VCD:6', 'X:VCD:7',
'X:VCD:8', 'X:VCD:9', 'X:Glc:1', 'X:Glc:2', 'X:Glc:3', 'X:Glc:4', 'X:Glc:5',
'X:Glc:6', 'X:Glc:7', 'X:Glc:8', 'X:Glc:9']
```



Transfer learning over tasks A and B with one-hot encoding

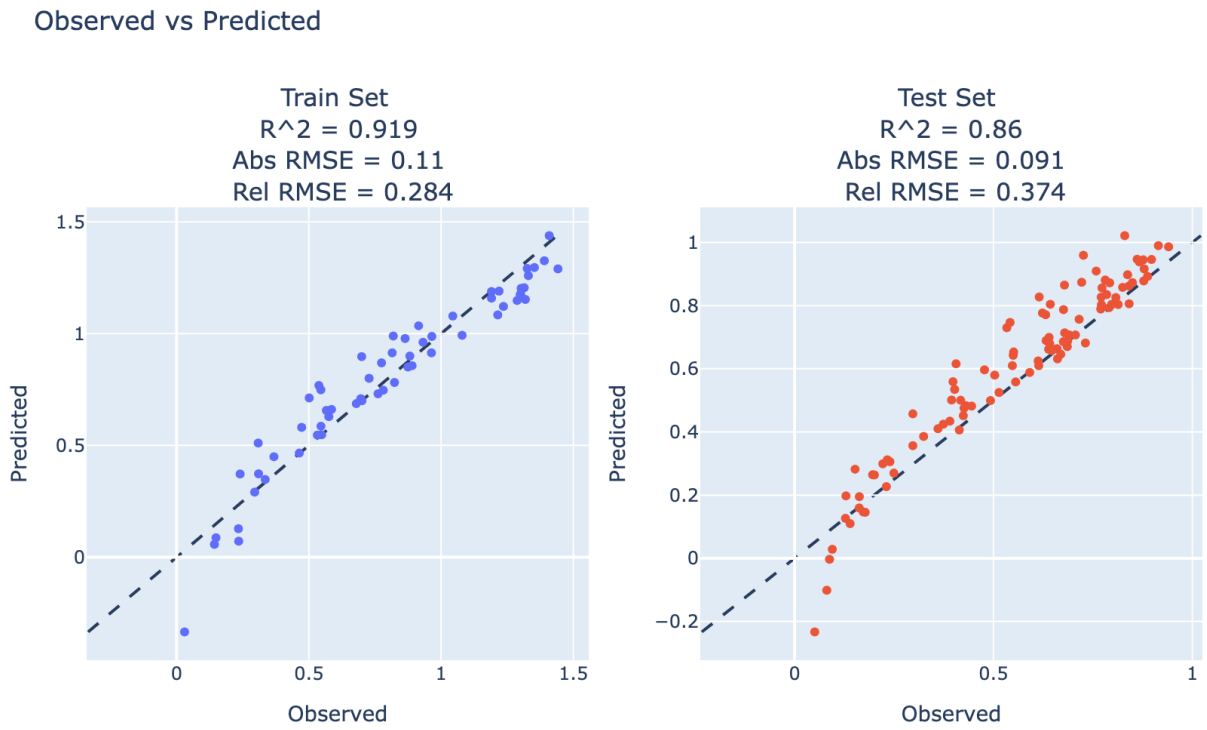
Now we add a task descriptor: a column `Task:B` that is 1 for task B experiments and 0 for task A experiments. The model can use this column to correct for the systematic difference between the processes.

```
# Define Pipeline
pscaler = StandardScaler(with_mean=USE_NORM,with_std=USE_NORM)
pls_bwu = PLSRegression(n_components=N_LV)
pipe = Pipeline([('scaler', pscaler), ('pls', pls_bwu)])
# Train PLS model
y = pd.concat([B_tar,A_tar])
X = pd.concat([preprocess_bwu(B_bwu, B_doe, USE_DAYS,
REMOVE_VARS),preprocess_bwu(A_bwu, A_doe, USE_DAYS, REMOVE_VARS)],axis=0)
X['Task:B'] =
np.concatenate((np.repeat(1,len(B_tar)),np.repeat(0,len(A_tar))),axis=None)
X_columns = X.columns
print(len(X_columns), "input columns:", list(X_columns))
pipe.fit(X,y)

# Make predictions
yhat = pipe.predict(X)
X_test = preprocess_bwu(B_bwu_test, B_doe_test, USE_DAYS, REMOVE_VARS)
X_test['Task:B'] = np.repeat(1,len(B_tar_test))
y_test = B_tar_test
yhat_test = pipe.predict(X_test)

# Calculate the relative test error for the comparison at the end
scen_3 = round(root_mean_squared_error(y_test, yhat_test) / np.std(np.array(y_test)),
3)
# Plot observed vs predicted with error metrics (R2, absolute and relative RMSE)
plothelpers.observed_vs_predicted(y, yhat, y_test, yhat_test)
```

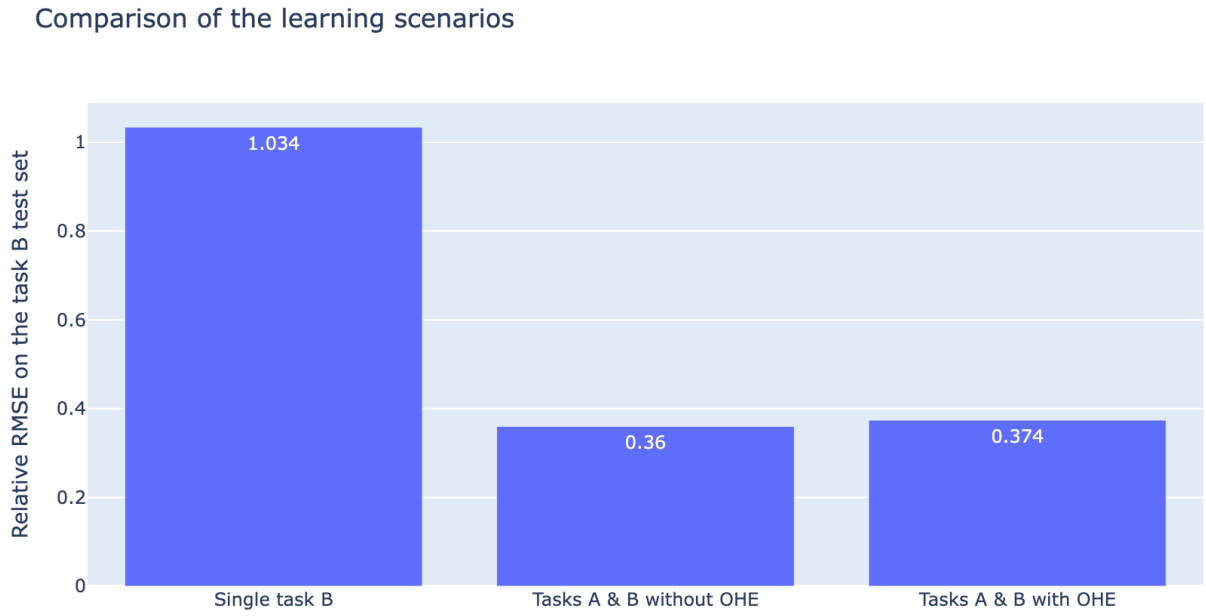
```
24 input columns: ['feed_start', 'feed_end', 'Glc_feed_rate', 'Glc_0', 'VCD_0',
'X:VCD:1', 'X:VCD:2', 'X:VCD:3', 'X:VCD:4', 'X:VCD:5', 'X:VCD:6', 'X:VCD:7',
'X:VCD:8', 'X:VCD:9', 'X:Glc:1', 'X:Glc:2', 'X:Glc:3', 'X:Glc:4', 'X:Glc:5',
'X:Glc:6', 'X:Glc:7', 'X:Glc:8', 'X:Glc:9', 'Task:B']
```



Comparison of the scenarios

The relative RMSE on the task B test set of the three models side by side.

```
# Comparison between the learning scenarios on the task B test set
fig = go.Figure([go.Bar(x=["Single task B", "Tasks A & B without OHE", "Tasks A & B
with OHE"], y=[scen_1, scen_2, scen_3], text=[scen_1, scen_2,
scen_3],textposition='auto',)])
fig.update_layout(yaxis=dict(title='Relative RMSE on the task B test set'),
title='Comparison of the learning scenarios')
fig.show()
```



Tasks

1. Change the number of task B training experiments `NUM_RUNS` to 5 and to 30. At which point does the single-task model catch up with the transfer learning models?
2. Make task B more different from task A (for example `K_I_LAC = 10` and `K_GLC = 0.08`). Does pooling the data without the task descriptor still help?
3. Change the number of latent variables `N_LV`. Is the optimal number the same for the single-task and the transfer learning models?

```
# Your code here
```